Unit–2 Assignment: Linear Data Structures

Course Title: Data Structures

Course Code: ETCCDS202

Unit Number: 2

Unit Title: Linear Data Structures

Submitted To: Ms Neetu Chauhan

Student Name: Himalaya Singh

Roll No.: 2501730154

Section: A

Submission Date: 17-04-2026

Academic Year : 2025-2029

Dynamic Array – Amortized Analysis

In a dynamic array, elements are stored in a continuous block of memory. When we insert elements using the append() operation, most of the time the element is directly added at the end, which takes constant time.

However, when the array becomes full, we need to create a new array with larger capacity (usually double the size) and copy all existing elements into it. This copying takes linear time.

Even though resizing is costly, it does not happen frequently. Because of this, the overall cost of multiple append operations gets distributed. So, the average time per append operation is still constant.

Conclusion:

Most appends → O(1)

Resizing step → O(n)

Overall (amortized) → O(1)

Stack Implementation (Using Singly Linked List)

A stack follows the Last In First Out (LIFO) principle. In this implementation, insertion and deletion are done at the head of the linked list to make operations efficient.

Time Complexity:

push(x) → O(1) (insert at head) pop()

→ O(1) (remove from head) peek()

→ O(1) (access head element)

Since no shifting of elements is required, all operations take constant time.

Queue Implementation (Using Singly Linked List)

A queue follows the First In First Out (FIFO) principle. We maintain both head and tail pointers to ensure efficient operations.

Time Complexity:

enqueue(x) → O(1) (insert at tail) dequeue()

→ O(1) (remove from head) front() → O(1)

(access head element)

Using both head and tail avoids traversal, making operations fast.

Balanced Parentheses Checker (Using Stack)

This problem checks whether an expression has properly matched brackets.

Working Idea:

Traverse the expression character by character

Push opening brackets onto the stack

When a closing bracket appears, check if it matches the top of the stack

If mismatch occurs or stack is empty, expression is not balanced

At the end, if stack is empty → balanced

Examples:

"([])" → Balanced

"([)]" → Not Balanced

"((((" → Not Balanced

"" → Balanced

This shows how a stack can be used to solve real problems involving order and matching.

Conclusion

In this assignment, we implemented linear data structures like dynamic arrays, linked lists, stacks, and queues from scratch. It helped in understanding how data is stored and managed internally without relying on built-in functions. We also applied these concepts to solve a real-world problem using stacks.

References

- GeeksforGeeks
- W3Schools